

Performance and Privacy: A Low-Latency Secure Anonymous Authentication Protocol with OPRF

Wenjv Hu¹, Yanping Ye^{2(✉)} and Yin Li^{1(✉)}

¹ School of Computer Science and Technology, Dongguan University of Technology, Dongguan, China

² Dongguan Sixth People's Hospital, China
793615497@qq.com
liyini@dgut.edu.cn

Abstract. Performing privacy-preserving queries, particularly anonymous authentication, against large-scale datasets presents critical tradeoffs between security, latency, scalability. Existing cryptographic solutions often impose linear computation or communication overheads. This paper introduces a novel, efficient protocol for secure anonymous authentication, uniquely combining matrix partitioning via hash prefixes with Oblivious Pseudorandom Functions in a three-server semi-honest model. Crucially, compared to our previous work published at TrustCom 2024, this enhanced protocol eliminates the dependency on a designated fully trusted server, achieving security when any single server is corrupted. Furthermore, our protocol demonstrates significant performance improvements over current state-of-the-art methods. It achieves sub-linear online communication complexity. Evaluations show that for datasets of size $m \approx 10^6$, our protocol reduces online communication by at least 30% compared to other sub-linear schemes, while maintaining competitive online computation times. Security is proven via simulation, and comprehensive experiments confirm practicality for datasets up to $m = 10^8$.

Keywords: Privacy-preserving authentication, Oblivious pseudorandom function, Bloom filter.

1 Introduction

As the pace of digitalization accelerates, the volume of data accumulated across various sectors is growing at an unprecedented rate. From financial transaction records and medical health archives to user behavior logs, these massive datasets hold immense value but simultaneously pose severe challenges to privacy protection [1]. Particularly in scenarios requiring anonymous authentication, effectively protecting user query privacy and the data privacy of data owners while leveraging data has become a critical issue. Existing approaches addressing privacy-preserving queries can be broadly classified into three main categories:

Private Set Intersection (PSI) protocols allow parties to securely compute the intersection of their private sets. For example the high-throughput two-party semi-honest

PSI using OKVS and VOLE by Raghuraman and Rindal [2], and optimized FHE-based approaches for unbalanced PSI by Cong et al. [3]. Kavousi et al. [4] employed oblivious transfer extensions and obfuscated Bloom filters to enhance scalability, although efficiency can degrade with highly unbalanced datasets characteristic of client-server authentication scenarios. Other works, such as Niu et al. [5], focus on addressing resource-constrained clients but may increase protocol complexity for specific operations like threshold checks. Multi-party PSI (MPSI) schemes [6, 7] utilizing tools like Bloom filters and threshold encryption have also been explored for collaborative computation scenarios.

Private Information Retrieval (PIR) enables a client to retrieve a database entry without revealing the index to the server. Progress includes throughput enhancements in FHE-based PIR (KsPIR [8]), schemes achieving near-optimal amortized costs via client preprocessing (APRS-based work by Lazzaretti and Papamanthou [9]), stateful approaches (FrodoPIR [10]), and explorations into relaxed privacy models (WPIR [11]). Piano [12], a simple single-server scheme, achieves sublinear $O(\sqrt{n})$ online server computation but requires substantial $O(\sqrt{n})$ client storage and necessitates clients downloading the database during preprocessing, which is impractical for large datasets. Other single-server PIR protocols, including Spiral [13] and OnionPIR [14], leverage techniques such as lattice-based homomorphic encryption to improve efficiency, particularly communication. However, these advanced cryptographic primitives often introduce significant computational overhead.

Oblivious Pseudorandom Functions (OPRF) [15] allow a client to learn a PRF evaluation on its input using the server's key, without revealing the input to the server or the key to the client. Advancements span practical lattice-based VOPRFs [16], iOPRF for structured data [17], and combinations with Bloom filters or HE for applications like approximate membership testing [18].

However, despite these advancements, applying these scheme directly to achieve efficient, large-scale, low-latency anonymous authentication presents challenges, largely falling into the following categories:

- **Performance Scalability:** A primary bottleneck remains the linear scaling of online server computation or communication costs with dataset size m . This affects even highly optimized PSI protocols [2], FHE-based PIR schemes [3], and naive applications of OPRF, hindering their practicality for real-time responses on massive datasets.
- **Functional Mismatch and Inefficiency:** There is often a mismatch between the functionality provided and the specific needs of anonymous authentication. The full data retrieval capability offered by PIR techniques [8,9,10] is inherently excessive for simple existence checks, leading to unnecessary overhead. Similarly, complex cryptographic machinery used in some MPSI protocols [6,7], while powerful for general computation, adds undue complexity for basic membership verification.
- **Client-Side Burden:** Certain approaches impose substantial resource requirements on the client. Notably, some advanced PIR constructions [9,10] necessitate significant client storage or extensive preprocessing, rendering them unsuitable for lightweight or stateless clients prevalent in many authentication scenarios.

In our prior work [19], we introduced an efficient protocol for secure anonymous queries using data partitioning and Garbled Bloom Filters. However, a significant limitation of that protocol was its security reliance on one of the servers (Server3) being fully honest and not corrupted. This assumption, while simplifying the design, may not hold in all practical distributed environments and restricts the protocol’s deployment scenarios.

To overcome this limitation, this paper proposes a novel secure anonymous authentication protocol designed to operate under the standard semi-honest model where any one of the three participating servers can be corrupted. We achieve this enhanced security guarantee by fundamentally changing the result verification mechanism, replacing the Garbled Bloom Filter approach with one based on Oblivious Pseudorandom Functions (OPRF) combined with standard Bloom Filters, while retaining the efficient matrix partitioning structure via hash prefixes. Our core contribution lies in designing this new Protocol that achieves the following key features:

- **Novel Verification Mechanism:** Designed a new verification phase combining Oblivious Pseudorandom Functions with standard Bloom Filters, offering an efficient alternative to prior approaches based on Garbled Bloom Filters.
- **Relaxed Trust Assumption:** Eliminated the dependency on a fully trusted server required by our previous scheme [19], achieving security in the standard semi-honest model where any single server can be corrupted.
- **State-of-the-Art Performance:** Demonstrated significant performance advantages over relevant state-of-the-art protocols through experiments. Notably, the protocol achieves sub-linear online communication complexity. At $m \approx 10^6$, this results in **at least 30% less online communication** compared to other sub-linear schemes.

2 Preliminaries

Oblivious Transfer (OT). Oblivious Transfer (OT) [20] is a fundamental cryptographic protocol that allows a Sender, holding multiple messages, to interact with a Receiver such that the Receiver can choose one message to receive while satisfying the following security properties:

- **Receiver Privacy:** The Sender does not learn which message the Receiver chose.
- **Sender Privacy:** The Receiver only obtains the message they selected and learns nothing about the other messages.

Secret Sharing. Secret sharing [21] is a technique for dividing a secret s into multiple shares such that the original secret can only be reconstructed when a sufficient number of shares are combined. This paper employs an additive secret sharing scheme over a finite field $\mathbb{Z}_p$ (where p is a large prime):

- A secret $s \in \mathbb{Z}_p$ is split into n random shares $s_1, s_2, \dots, s_n \in \mathbb{Z}_p$ such that $s = (s_1 + s_2 + \dots + s_n) \pmod{p}$.

- Every secret share is characterized by uniform randomness, thereby imparting no information concerning the secret s .
- Only by collecting all n shares can the secret s be recovered.

Oblivious Pseudorandom Function (OPRF). An Oblivious Pseudorandom Function (OPRF) [22] is a two-party protocol that allows one party (the client) to obtain the output of a pseudorandom function $F(k, x)$, where k is a key held by the other party (the server) and x is the client's input. The protocol ensures:

- The server does not learn the client's input x .
- The client does not learn the key k and only obtains the output for the single value $F(k, x)$.

Bloom Filter. A Bloom filter [23] is a space-efficient probabilistic data structure used to test whether an element is a member of a set. It consists of an m -bit array, initially all set to 0, and k independent hash functions $h_1, \dots, h_k$.

- **Insertion:** To add an element x to the filter, for $j = 1$ to k , compute the index $\text{idx}_j(x) \pmod{m}$ and set the bit $BF[\text{idx}_j] = 1$.
- **Query:** To test if an element y is in the set, for $j = 1$ to k , compute the index $\text{idx}_{j'} = h_j(y) \pmod{m}$ and check the bit $\text{idx}_{j'}$. If any $\text{idx}_{j'}$ is 0, then y is definitively not in the set. If all $\text{idx}_{j'}$ are 1, then y is *possibly* in the set.

3 Protocol Description

This section elaborates on our proposed efficient secure anonymous authentication protocol. The protocol primarily consists of three phases, as illustrated in Figure 1.

Table 1. Main Notations and Meanings Symbol Meaning

Symbol	Meaning
m	Size (number of elements) of the dataset X
x	Client's query element
p	Prime modulus for the finite field $\mathbb{Z}_p$
n	Hash prefix length (in bits)
h_{pre}	Query prefix: first n bits of h
k	Number of rows in M (number of unique prefixes)
j	Number of columns in M (max row length after padding)
W	Unit vector of length k representing the query row index b
sk_{oprf}	Secret key for the OPRF held by <i>Server3</i>
A, B	Blinded elements exchanged during the OPRF protocol
$\text{idx}_{j'}$	Index $h_{j'}(V_q) \pmod{t}$ computed for BF query

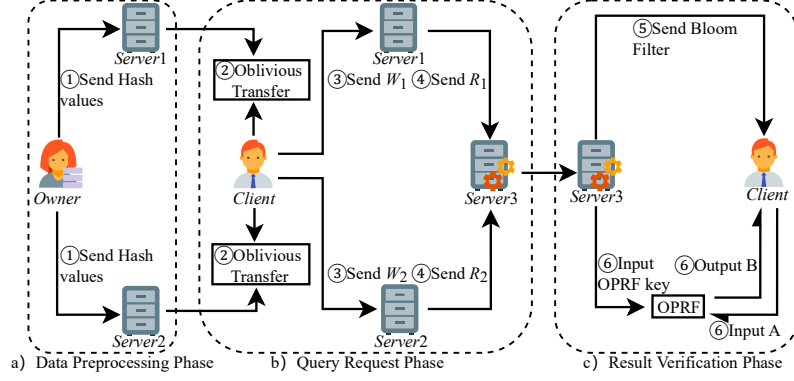


Fig. 1: Overall Scheme Flowchart.

3.1 Detailed Protocol Design

Fundamental Settings: All primary arithmetic operations are performed in a finite field $\mathbb{Z}_p$, where p is a sufficiently large prime number to ensure cryptographic security and accommodate the output values of all hash functions.

Phase 1: Data Preprocessing. Data Owner(S) applies a cryptographic hash function h_0 to each element in its original dataset $X = \{x_1, \dots, x_m\}$, obtaining the hashed dataset $Y = \{y_1, \dots, y_m\}$, where $y_i = h_0(x_i)$. S securely transmits Y to both Server1 and Server2. The goal is to select a prefix length n to balance the matrix dimensions (k rows $\times$ j columns). A primary strategy is based on the dataset size m :

$$n \leftarrow \lceil \log_2(\sqrt{m}) \rceil \quad (1)$$

The theoretical basis for this strategy is that computational and communication complexities can be optimally balanced when the number of rows ($k \approx \sqrt{m}$) and columns ($j \approx \sqrt{m}$) in the matrix are approximately equal, aiming for sub-linear complexity. In practice, n should be chosen based on the specific context.

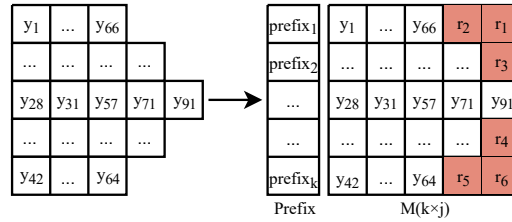


Fig. 2: Data Partitioning and Random Padding Illustration

Algorithm 1 Simplified Hash Prefix Partitioning and Matrix Generation

Input: Hash dataset $Y = \{y_1, \dots, y_m\}$, prefix length n

Output: Data matrix M , prefixes $PrefixList$

```

1:  $PrefixMap \leftarrow$  empty map
2:  $maxLength \leftarrow 0$ 
3: for all  $h$  in  $Y$  do
4:    $prefix \leftarrow$  first  $n$  bits of  $h$ 
5:   if  $prefix$  not in  $PrefixMap$  then
6:      $PrefixMap[prefix] \leftarrow []$ 
7:   end if
8:   Append  $h$  to  $PrefixMap[prefix]$ 
9:    $currentLength \leftarrow |PrefixMap[prefix]|$ 
10:   $maxLength \leftarrow \max(maxLength, currentLength)$ 
11: end for
12:  $PrefixList \leftarrow Keys(PrefixMap)$ 
13:  $k \leftarrow |PrefixList|$ 
14:  $j \leftarrow maxLength$ 
15: Initialize  $M$  as a  $k \times j$  matrix
16: for  $i \leftarrow 0$  to  $k - 1$  do
17:    $currentPrefix \leftarrow PrefixList[i]$ 
18:    $hashesForPrefix \leftarrow PrefixMap[currentPrefix]$ 
19:    $numHashes \leftarrow |hashesForPrefix|$ 
20:   for  $col \leftarrow 0$  to  $j - 1$  do
21:     if  $col < numHashes$  then
22:        $M[i, col] \leftarrow hashesForPrefix[col]$ 
23:     else
24:        $r \leftarrow RandomElement(\mathbb{Z}_p)$ 
25:        $M[i, col] \leftarrow r$ 
26:     end if
27:   end for
28: end for
29: return  $M, PrefixList$ 

```

Execute Algorithm 1. Partition Y based on the first n bits of each hash y_i . To ensure all rows have the same length j , padding is performed using random values uniformly selected from $\mathbb{Z}_p$. This generates the data matrix $M(k \times j)$ and the list of prefixes ‘PrefixList’, as illustrated in Figure 2. Both *Server1* and *Server2* hold M and PrefixList.

For the k prefixes in PrefixList, *Server1* and *Server2* associate each prefix $prefix_i = PrefixList[i]$ with its unique row index. They then use a symmetric encryption algorithm E (e.g., AES) with the prefix itself as the key to encrypt the corresponding row index, constructing the encrypted prefix index list $EPrefixList = \{E(prefix_i, i)\}_{i=0}^{k-1}$. *Server1* holds $EPrefixList$.

Phase 2: Query Request. Client(Q) computes the hash value $h = h_0(x)$ of its query element x and extracts its first n bits as the query prefix h_{pre} . Q randomly selects one server S_x from $\{Server1, Server2\}$ to interact with for the index acquisition step. (Assume $S_x = Server_1$ for the description below).

Server1 sends its copy of *EPrefixList* to *Q*. *Q* attempts to decrypt each ciphertext $C_i = E(\text{prefix}_i, i)$ in *EPrefixList* using its own h_{pre} as the key. If *Q* successfully decrypts some C_i and verifies that the decrypted prefix matches h_{pre} , then *Q* obtains the corresponding row index $b = i$. If *Q* fails to decrypt any entry corresponding to h_{pre} , it indicates x is definitively not in the dataset. The protocol terminates here for this query.

Q constructs a unit vector W of length k , where $W[b] = 1$ and all other elements are 0. Subsequently, *Q* generates a random vector K of length k , with elements chosen uniformly at random from $\mathbb{Z}_p$. *Q* computes the additive secret shares of W : $W_1 = K$ and $W_2 = (W - K) \pmod{p}$. *Q* sends W_1 to *Server1* and W_2 to *Server2*. *Server1* and *Server2* locally perform distributed matrix-vector multiplication upon receiving their respective shares W_1 and W_2 . This matrix-vector multiplication step is executed using the procedure detailed in our previous work [19]. *Server1* sends the computed R_1 to *Server3*. *Server2* sends the computed R_2 to *Server3*.

Phase 3: Result Verification. *Server3* first reconstructs the target row vector $R = (R_1 + R_2) \pmod{p}$ from the received additive shares. This vector R contains the set of hash values and padding elements corresponding to the client *Q*'s query prefix. *Server3* then generates its secret OPRF key, denoted sk_{oprf} , and proceeds to evaluate the Oblivious Pseudorandom Function for each element $r_i \in R$. These resulting OPRF values $\{V_i\}$ are subsequently inserted into a Bloom Filter (BF) structure of size t using l independent hash functions $(h_1, \dots, h_l)$. Upon completion of the BF construction, *Server3* transmits the populated BF to the client *Q*.

To obtain the OPRF evaluation corresponding to its own query x , *Q* engages interactively with *Server3* executing the OPRF protocol. Specifically, *Q* computes its query hash $h = h_0(x)$, maps it to a group element $P = H(h)$, selects a random blinding factor α from the appropriate domain (e.g., $\mathbb{Z}_q^*$ where q is the group order), computes the blinded element $A = P^\alpha$, and transmits A to *Server3*. Possessing the secret key sk_{oprf} , *Server3* computes the blinded evaluation $B = A^{sk_{\text{oprf}}} = (P^\alpha)^{sk_{\text{oprf}}}$ and returns B to *Q*. Upon receiving B , *Q* removes the blinding by computing the modular inverse $\alpha^{-1} \pmod{q}$ and calculating $V_q = B^{\alpha^{-1}} = (P^{sk_{\text{oprf}}})^{\alpha^{-1}}$. This yields the desired OPRF value $V_q = F(sk_{\text{oprf}}, h)$ obliviously, ensuring that *Server3* gains no information about *Q*'s input h .

For the final verification step, *Q* determines whether V_q indicates membership by querying the received Bloom Filter BF. *Q* computes the l relevant indices $\text{idx}_{j'} = h_{j'}(V_q) \pmod{t}$ using the same hash functions $h_1, \dots, h_l$ employed by *Server3*. If all corresponding bit positions $\text{BF}[\text{idx}_{j'}]$ are set to 1, *Q* concludes that its query x exists in the original dataset. Conversely, if any bit $\text{BF}[\text{idx}_{j'}]$ is found to be 0, *Q* definitively concludes that x does not exist. For readers interested in a step-by-step walkthrough, an illustrative example detailing the protocol's flow is provided as supplementary material online¹, as it could not be included in this version due to space constraints.

¹ For examples and a detailed security analysis see: <https://drive.google.com/file/d/1HnkGw4Ma2lmXIQspvbiz9NP2qCsfqlUX/view?usp=sharing>

4 Security Analysis

We construct a probabilistic polynomial-time (PPT) simulator S_P for each potentially corrupted party $P \in \{Q, \text{Server}_1, \text{Server}_2, \text{Server}_3\}$. Given P 's input input_P and output output_P , S_P must generate a view that is computationally indistinguishable from the actual view VIEW_P^π obtained by P in a real protocol execution π . The view VIEW_P^π encompasses P 's input, internal random coins, and all received messages. Formally, we must demonstrate that for security parameter κ :

$$\{S_P(\text{input}_P, \text{output}_P)\}_\kappa \stackrel{c}{\equiv} \{\text{VIEW}_P^\pi(\text{inputs}, \text{randomness})\}_\kappa \quad (2)$$

where $\stackrel{c}{\equiv}$ denotes computational indistinguishability. This demonstrates that the protocol execution reveals no more information to P than what is inherently derivable from its designated inputs and outputs.

Theorem *Server data privacy is preserved against the semi-honest client Q . Client query privacy is protected against semi-honest Servers 1 and 2. Furthermore, when the client interacts with a semi-honest Server 3, the protocol not only upholds this client query privacy but also critically extends its safeguards to ensure the confidentiality of the client's OPRF key.*

Security Summary: The existence of simulators for each potential single corrupted party demonstrates that the protocol is secure in the semi-honest model. No party learns more information than intended by the protocol specification (i.e., their inputs and outputs). Crucially, unlike our prior work [19], this security guarantee holds even if *Server3* is corrupted, significantly relaxing the trust assumptions by removing the need for any fully trusted server. The security relies on the standard assumption that no two servers collude. For brevity, this section formalizes only the security model and theorem statements. Detailed simulator constructions, hybrid argumentation processes, and full security reductions are provided of the full version of this paper^{1 above}. The complete proof includes per-party simulator designs, message indistinguishability analyses across protocol phases, and a sequence of hybrid games formalizing security against semi-honest adversaries.

5 Performance Evaluation

5.1 Theoretical Complexity Analysis

We analyze the asymptotic complexity of the protocol. Let m be the dataset size, k the number of matrix rows, j the number of columns (with $k, j \approx \sqrt{m}$), t the Bloom filter size, and l the number of BF hash functions.

Preprocessing Phase: This phase is dominated by the data owner hashing m elements ($O(m)$ work) and the servers partitioning the data and constructing the $k \times j$ matrix M (typically $O(m \log m)$ or $O(m)$ depending on implementation). Offline communication from the owner to servers is $O(m)$.

Online Query Phase:

- *Computation:* The client performs $O(k)$ work for the OT interaction and $O(l)$ for BF verification. *Server1* or *Server2* computation is dominated by the matrix-vector multiplication, requiring $O(kj) \approx O(m)$ operations. *Server3* reconstructs the row, computes $O(j)$ OPRF values, and inserts them into the Bloom filter. The overall online computational bottleneck is the $O(m)$ matrix multiplication performed distributively by *Server1* and *Server2*. Cryptographic operations on the client and *Server3* scale approximately with $O(K + l)$ and $O(j \cdot l)$ respectively.
- *Communication:* The total online communication complexity is $O(K + j + t)$. Since $k, j \approx \sqrt{m}$ and the Bloom filter size t is typically chosen based on j and the desired error rate ϵ (often sub-linear in m , e.g., $t \approx O(\sqrt{m} \log(1/\epsilon))$), the overall online communication complexity is dominated by $O(\sqrt{m})$.

Summary: Our protocol achieves $O(\sqrt{m})$ online communication complexity, which is highly advantageous for large datasets. This sub-linear online communication comes at the cost of an amortizable $O(m \log m)$ offline computation phase and an $O(m)$ online computation bottleneck.

5.2 Implementation Details

Our protocol, implemented in Python, achieves approximately 128-bit security using a 256-bit prime field compatible with elliptic curve operations. Key cryptographic components include SHA-256 for the initial hash h_0 , Curve25519 for OPRF computations with appropriate hash-to-curve methods, and AES-128 in CTR mode for symmetric encryption of dynamically determined prefix identifiers. Bloom filter parameters were set for a false positive rate below 10^{-9} , utilizing $k = 7$ hash functions derived from SHA-256.

Experiments were run on two identical Windows 10 Pro machines (AMD Ryzen 7 7700, 32GB RAM) connected via a 1 Gbps LAN (sub-millisecond RTT). One machine hosted Client(Q) and Server3, the other Server1 and Server2. We measured end-to-end wall-clock times and network data for random 128-bit identifier datasets of sizes $m \in \{10^5, 10^6, 10^7, 10^8\}$, averaging over 10 queries per size. For comparison, state-of-the-art protocols were benchmarked on our testbed using their public implementations.

Performance results (Figure 3) indicate that offline preprocessing time and communication scale roughly linearly with dataset size m . For $m = 10^8$, these reached 719 seconds and 3.21 GB, respectively.

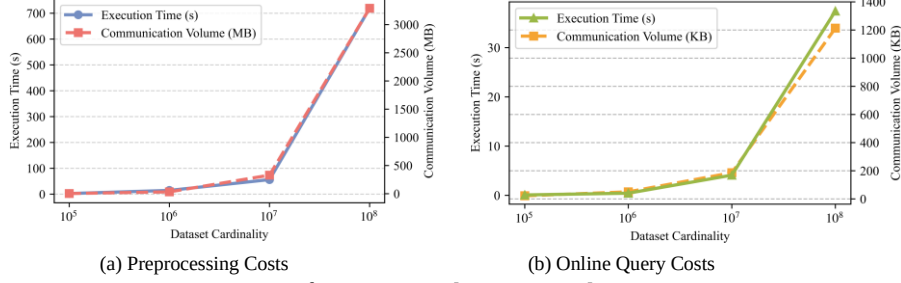


Fig. 3: Performance Evaluation Results.

The online performance (Figure 3b) highlights the protocol's effectiveness for large datasets. Online communication scales sub-linearly, consistent with the theoretical $O(\sqrt{m})$ complexity, measuring 50.22 KB for $m = 10^6$ and 1.18 MB for $m = 10^8$. Online computation time increases from 0.44s ($m = 10^6$) to 37.43s ($m = 10^8$), demonstrating practical feasibility.

Table 2. Performance Comparison with State-of-the-Art Protocols

Protocol	Functionality	Online Time(s) (per query)	Online Comm (Total, per query)	Online Comm Complexity	Offline Cost (Preprocessing)
This Work	Authentication	0.44	50.22 KB	$O(\sqrt{m})$	Yes(Server $O(m \log m)$)
Blazing Fast PSI	2P-PSI	0.16	22.5 MB	$O(m \log m + m\lambda)$	No
PIANO	1S-PIR	0.45	70.6 KB	$O(\sqrt{m})$	Yes(Client $O(m)$)
FrodoPIR	1S-PIR	0.83	4 MB	$O(m)$ Query	Yes(Srv+Client)

A comparison with state-of-the-art protocols (Blazing Fast PSI [2], FrodoPIR [10], PIANO [12]) for $m \approx 10^6$ (Table 2) reveals:

Online Time Analysis: Our protocol's online query time of 0.44 seconds is competitive, nearly twice as fast as FrodoPIR (0.83s) and similar to PIANO (0.45s). While specialized two-party PSI like Blazing Fast PSI is faster (0.16s), our multi-server model is suited for authentication without direct client-server PSI on full dataset elements and offers comparable speed to relevant PIR schemes.

Online Communication Analysis: Our protocol excels in online communication efficiency, requiring only 50.22 KB per query. This is significantly lower (approx. 80x less than FrodoPIR, over 450x less than Blazing Fast PSI) than protocols with linear or near-linear communication. It also shows superior efficiency (approx. 30% less communication) compared to PIANO, another $O(\sqrt{m})$ protocol, due to our matrix partitioning and efficient secure computations.

Offline Costs and Model Considerations: Our protocol has a one-time/infrequent offline preprocessing phase with $O(m \log m)$ computation and $O(m)$ communication for servers, amortizable for datasets not changing too rapidly. PIANO also has an $O(m)$ offline phase but involves client downloading database-dependent information. FrodoPIR also requires preprocessing, while Blazing Fast PSI does not.

Summary of Comparison: Our protocol offers a compelling balance, competitive online query times comparable to efficient PIR schemes, and significantly superior online communication overhead, outperforming even other sub-linear schemes. This makes it well-suited for large-scale secure anonymous authentication where minimizing per-query costs is critical.

6 Conclusion

Addressing the performance limitations of existing privacy-preserving techniques for large-scale anonymous authentication, this paper presented a novel protocol combining matrix partitioning with OPRF in a three-server architecture. Our protocol successfully achieves sub-linear online communication complexity ($O(\sqrt{m})$) while maintaining competitive online computation times and provable security in the semi-honest model. Experimental results validate the practicality and significant communication efficiency gains of our approach compared to prior art, demonstrating its suitability for deploying low-latency, privacy-preserving authentication services over massive datasets. This work offers a concrete step towards bridging the gap between the demand for data privacy and the performance requirements of large-scale real-world systems.

Acknowledgments. This work is supported by National Natural Science Foundation of China (Grant no. 62372107) and National Key Research and Development Program of China (Grant 2021YFB3101303)

References

1. Liu, L., Li, J., Lv, J., Wang, J., Zhao, S., Lu, Q.: Privacy-preserving and secure industrial big data analytics: A survey and the research framework. *IEEE Internet of Things Journal* 11(11), 18976–18999 (2024)
2. Raghuraman, S., Rindal, P.: Blazing fast psi from improved okvs and subfield vole. In: *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. pp. 2505–2517 (2022)
3. Cong, K., Moreno, R.C., da Gama, M.B., Dai, W., Iliashenko, I., Laine, K., Rosenberg, M.: Labeled psi from homomorphic encryption with reduced computation and communication. In: *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. pp. 1135–1150 (2021)
4. Kavousi, A., Mohajeri, J., Salmasizadeh, M.: Efficient scalable multi-party private set intersection using oblivious prf. In: *Security and Trust Management: 17th International Workshop, STM 2021, Darmstadt, Germany, October 8, 2021, Proceedings* 17. pp. 81–99. Springer (2021)
5. Ma, L., Wang, H., Niu, Z., Li, Z., Wu, L., Wei, X., Su, Y.: Over-threshold multiparty private set operation protocols for lightweight clients. *Computer Standards & Interfaces* 88, 103781 (2024)
6. Bay, A., Erkin, Z., Hoepman, J.H., Samardjiska, S., Vos, J.: Practical multi-party private set intersection protocols. *IEEE Transactions on Information Forensics and Security* 17, 1–15 (2021)

7. Debnath, S.K., Stănică, P., Kundu, N., Choudhury, T.: Secure and efficient multi-party private set intersection cardinality (2021)
8. Luo, M., Liu, F.H., Wang, H.: Faster fhe-based single-server private information retrieval. In: *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. pp. 1405–1419 (2024)
9. Lazzaretti, A., Papamanthou, C.: Near-optimal private information retrieval with preprocessing (2022)
10. Davidson, A., Pestana, G., Celi, S.: FrodoPir: Simple, scalable, single-server private information retrieval. *Proceedings on Privacy Enhancing Technologies* (2023)
11. Lin, H.Y., Kumar, S., Rosnes, E., i Amat, A.G., Yaakobi, E.: Multi-server weaklyprivate information retrieval. *IEEE Transactions on Information Theory* 68(2), 1197–1219 (2021)
12. Zhou, M., Park, A., Zheng, W., Shi, E.: Piano: extremely simple, single-server pir with sub-linear server computation. In: *2024 IEEE Symposium on Security and Privacy (SP)*. pp. 4296–4314. IEEE (2024)
13. Menon, S.J., Wu, D.J.: Spiral: Fast, high-rate single-server pir via fhe composition. In: *2022 IEEE symposium on security and privacy (SP)*. pp. 930–947. IEEE (2022)
14. Mughees, M.H., Chen, H., Ren, L.: Onionpir: Response efficient single-server pir. In: *Proceedings of the 2021 ACM SIGSAC conference on computer and communications security*. pp. 2292–2306 (2021)
15. Freedman, M.J., Ishai, Y., Pinkas, B., Reingold, O.: Keyword search and oblivious pseudorandom functions. In: *Theory of Cryptography: Second Theory of Cryptography Conference, TCC 2005, Cambridge, MA, USA, February 10-12, 2005. Proceedings 2*. pp. 303–324. Springer (2005)
16. Albrecht, M.R., Gur, K.D.: Verifiable oblivious pseudorandom functions from lattices: Practical-ish and thresholdisable. In: *International Conference on the Theory and Application of Cryptology and Information Security*. pp. 205–237. Springer (2024)
17. Blass, E.O., Kerschbaum, F., Mayberry, T.: Iterative oblivious pseudo-random functions and applications. In: *Proceedings of the 2022 ACM on asia conference on computer and communications security*. pp. 28–41 (2022)
18. Stanciu, V.D., Steen, M.v., Dobre, C., Peter, A.: Privacy-preserving crowdmonitoring using bloom filters and homomorphic encryption. In: *Proceedings of the 4th International Workshop on Edge Systems, Analytics and Networking*. pp. 37–42 (2021)
19. Hu, W., Li, Y.: An efficient and secure anonymous query protocol. In: *2024 IEEE 23rd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. pp. 2151–2158. IEEE (2024)
20. Rabin, M.O.: How to exchange secrets with oblivious transfer. *Cryptology ePrint Archive* (2005)
21. Boyle, E., Gilboa, N., Ishai, Y.: Function secret sharing: Improvements and extensions. In: *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*. pp. 1292–1303 (2016)
22. Freedman, M.J., Ishai, Y., Pinkas, B., Reingold, O.: Keyword search and oblivious pseudorandom functions. In: *Theory of Cryptography: Second Theory of Cryptography Conference, TCC 2005, Cambridge, MA, USA, February 10-12, 2005. Proceedings 2*. pp. 303–324. Springer (2005)
23. Naor, M., Yaguev, E.: Bloom filters in adversarial environments. In: *Annual Cryptology Conference*. pp. 565–584. Springer (2015)